

Linux Embebido – Técnicas Digitales III (TD3)

Módulo 1

De un RTOS a Linux: conceptos, historia
y puesta en marcha de un sistema headless

Autor: Ing. Marín Ariel

Auxiliares becarios
Fabrizio Carlassara (BIDOC)
Pablo Gómez (BIDOC)
Alexis Hernández de la Cruz (BIS)

Resumen

Este es el primer módulo del bloque de Linux embebido sobre Raspberry Pi. Tiene tres objetivos. Primero, entender de dónde viene Linux y por qué terminó siendo, hoy, el sistema operativo más usado en sistemas embebidos. Segundo, entender *conceptualmente* en qué se diferencia un sistema operativo de propósito general (GPOS, *General-Purpose Operating System*) de un sistema operativo de tiempo real (RTOS, *Real-Time Operating System*), que es el mundo del que vienen los alumnos. Tercero, dejar una Raspberry Pi funcionando en modo *headless* (sin monitor ni teclado), accesible por SSH, y dar los primeros pasos de navegación del sistema. Este entorno es la base sobre la que se construyen todos los módulos siguientes.

Índice

1. Objetivos del módulo	2
2. Un poco de historia: ¿de dónde viene Linux?	2
3. RTOS genérico vs. GPOS: ¿en qué se diferencian?	4
3.1. Determinismo: la pregunta central	4
3.2. Arquitectura: capas de abstracción, de la aplicación al hardware	4
3.3. Tabla comparativa	5
3.4. Políticas de scheduling, en más detalle	6
4. Los cuatro elementos de un sistema Linux embebido	7
5. Preparación del entorno: Raspberry Pi OS Lite headless	8
5.1. Paso 1: Grabar la imagen con Raspberry Pi Imager	8
5.2. Paso 2: Conectarse por SSH desde la PC	9
5.3. Paso 3: Primeros comandos de chequeo	9

6. El sistema de archivos de Linux	9
6.1. Permisos de archivos	9
6.2. Directorios más relevantes	9
7. Comandos esenciales de Linux	11
8. Cierre conceptual	12

1. Objetivos del módulo

Al finalizar este módulo, el alumno debería poder:

- Explicar brevemente de dónde viene Linux y por qué se usa en sistemas embebidos.
- Explicar, con sus propias palabras, qué distingue a un RTOS de un GPOS, tanto a nivel de determinismo como de arquitectura interna.
- Identificar y describir los cuatro elementos que componen cualquier sistema Linux embebido: toolchain, bootloader, kernel y sistema de archivos raíz.
- Instalar Raspberry Pi OS Lite en modo headless y conectarse por SSH sin usar monitor ni teclado.
- Navegar el sistema de archivos de Linux y entender el modelo de permisos de archivos.

2. Un poco de historia: ¿de dónde viene Linux?

Para entender por qué este curso usa Linux y no otra cosa, conviene conocer, con algo más de detalle, de dónde viene.

Los orígenes: Unix. Linux no nació de la nada. A fines de los años 60, en los laboratorios Bell de AT&T, Ken Thompson y Dennis Ritchie desarrollaron Unix, un sistema operativo pensado para minicomputadoras, con ideas que en su momento eran poco convencionales: un diseño modular de programas pequeños que se combinan entre sí, un sistema de archivos jerárquico simple, y –más adelante– la reescritura del propio sistema en un lenguaje de alto nivel (C, creado también por Ritchie para esa tarea) en lugar de en ensamblador. Eso hizo a Unix mucho más fácil de portar a otras arquitecturas de hardware que sus contemporáneos.

Durante los años 70, restricciones legales sobre AT&T (que en ese momento no podía comercializar software fuera de su negocio regulado de telefonía) llevaron a que Unix se distribuyera con su código fuente, casi sin costo, a universidades de todo el mundo. Eso generó una primera generación de programadores e investigadores que conocían el sistema a fondo, y sentó las bases de buena parte de la cultura de software libre que vendría después. Sin embargo, a comienzos de los 80, ese escenario cambió: AT&T empezó a licenciar Unix de forma comercial y restrictiva, y al mismo tiempo surgieron variantes propietarias incompatibles entre sí (System V de AT&T, BSD de Berkeley, y las versiones de cada fabricante de hardware: Sun, IBM, HP, DEC). El resultado fue un ecosistema fragmentado, con licencias pagas y código mayormente cerrado.

El Proyecto GNU. En 1983, en respuesta directa a esa fragmentación y al cierre del código, Richard Stallman inició el **Proyecto GNU** (*GNU's Not Unix*), con el objetivo de construir, desde cero, un sistema operativo completo, compatible con Unix, pero completamente libre. Para sostener ese objetivo a largo plazo, Stallman creó también la **GPL** (*General Public License*), una licencia de tipo *copyleft*: cualquiera puede usar, modificar y redistribuir el software, pero las

modificaciones que se distribuyan tienen que seguir siendo libres bajo la misma licencia. Esto evita que una empresa tome código libre, le agregue mejoras, y las vuelva a cerrar.

Durante los años 80, el proyecto GNU construyó, una por una, casi todas las piezas de un sistema operativo completo: un compilador de C (GCC), un shell, un editor de texto (Emacs), y decenas de utilidades de línea de comandos que cualquiera que haya usado Linux reconocería hoy (`ls`, `grep`, `cat`, etc.). Le faltaba, sin embargo, la pieza más difícil y central de cualquier sistema operativo: el kernel. El proyecto de GNU para esa pieza (llamado Hurd) avanzaba muy lentamente, con un diseño ambicioso basado en microkernel que tardó años en madurar.

El kernel de Linus Torvalds. Esa pieza faltante la aportó, casi por casualidad, un estudiante de la Universidad de Helsinki llamado Linus Torvalds. Torvalds usaba en su computadora Minix, un sistema operativo tipo Unix desarrollado por Andrew Tanenbaum con fines puramente educativos, y decidió escribir su propio kernel, en parte para entender mejor el procesador 80386 de su PC y en parte porque las limitaciones de Minix lo frustraban. En agosto de 1991, publicó en un grupo de noticias de Internet (Usenet, `comp.os.minix`) un mensaje hoy célebre, en el que anunciaba el proyecto describiéndolo –con una modestia que la historia no terminó de confirmar– como “solo un hobby, que no va a ser grande ni profesional como GNU”. En septiembre de ese año liberó la versión 0.01 del kernel, y en 1992 lo volvió a licenciar bajo la GPL.

Al estar bajo la GPL, ese kernel pudo combinarse naturalmente con las herramientas ya maduras del proyecto GNU (compilador, shell, utilidades), completando así el sistema operativo libre que GNU había estado construyendo durante toda la década anterior. A ese conjunto se lo empezó a llamar, de forma un poco informal, “Linux” (más precisamente, GNU/Linux, aunque esa precisión generó –y todavía genera– discusiones dentro de la comunidad sobre cómo nombrarlo correctamente).

De proyecto de estudiante a sistema operativo embebido dominante. ¿Por qué esto terminó siendo relevante para sistemas embebidos, que es lo que nos importa en este curso? Por varios motivos que se fueron reforzando entre sí con el tiempo:

- **Costo:** al ser libre y gratuito, sin costos de licencia por unidad fabricada, cualquier fabricante de hardware puede usarlo en su producto sin pagar regalías, algo importante cuando se fabrican millones de unidades de un dispositivo de bajo costo.
- **Código fuente disponible:** cualquier fabricante puede tomar el kernel, agregarle sus propios controladores para su hardware específico, y distribuirlo (la GPL exige que, si se distribuye, esas modificaciones al kernel también queden disponibles).
- **Soporte de arquitecturas:** a diferencia de muchos sistemas operativos comerciales de la época, Linux se portó relativamente pronto a una enorme variedad de arquitecturas de CPU (x86, ARM, MIPS, PowerPC, y más adelante RISC-V), justamente las que predominan en el mundo embebido.
- **Comunidad y ecosistema:** una comunidad enorme de desarrolladores –y, desde los 2000, empresas como IBM, Intel y Google, entre muchas otras– mantiene controladores para prácticamente cualquier periférico imaginable, y los aporta directamente al kernel oficial (*mainline*), lo que reduce drásticamente el trabajo de mantenimiento para cada fabricante.

El resultado es que Linux escaló desde routers domésticos hasta sistemas de infotainment automotriz, controladores industriales, electrodomésticos “inteligentes”, y por supuesto computadoras de placa única como la Raspberry Pi que vamos a usar en este curso. El ejemplo más masivo de todos es probablemente Android, que –aunque con una capa de aplicaciones muy distinta a la de una distribución de escritorio– usa el kernel de Linux como base, y hoy corre en miles de millones de dispositivos [1, cap. 1].

3. RTOS genérico vs. GPOS: ¿en qué se diferencian?

3.1. Determinismo: la pregunta central

La diferencia de fondo entre un RTOS y un GPOS no es “cuál es más rápido”, sino **cuán predecible** es el tiempo de respuesta del sistema ante un evento. Un sistema de tiempo real no es necesariamente veloz: es uno donde se puede garantizar un *tiempo máximo* de respuesta (un *deadline*) de forma determinística [1, cap. 21].

En un RTOS genérico (como el que los alumnos ya conocen de cursos anteriores), cuando una tarea de alta prioridad se desbloquea –por ejemplo, por una interrupción– el *scheduler* (planificador) la pone a correr casi inmediatamente, con una latencia acotada y conocida de antemano. En un GPOS como Linux, el kernel puede estar ejecutando código no interrumpible (una sección crítica, un manejador de interrupción, etc.) y la tarea de mayor prioridad puede tener que esperar. Esa espera variable se llama **latencia de scheduling** (*scheduling latency*), y es la métrica central cuando se habla de tiempo real en Linux [1, cap. 21].

Nota

Linux *puede* comportarse de forma mucho más determinística si se usa el kernel parcheado con PREEMPT_RT, que vuelve preemptible casi todo el código del kernel y convierte los manejadores de interrupción en hilos con prioridad asignable [1, cap. 21]. No es el foco de este curso, pero vale la pena saber que existe: es la vía por la cual Linux se usa en robótica, control industrial, etc.

3.2. Arquitectura: capas de abstracción, de la aplicación al hardware

La diferencia de determinismo viene, en gran parte, de una diferencia arquitectónica más profunda, que se entiende mejor mirando qué capas atraviesa el código hasta llegar al hardware en cada caso. La Figura 1 muestra esa pila de capas, lado a lado, para un RTOS genérico y para Linux.

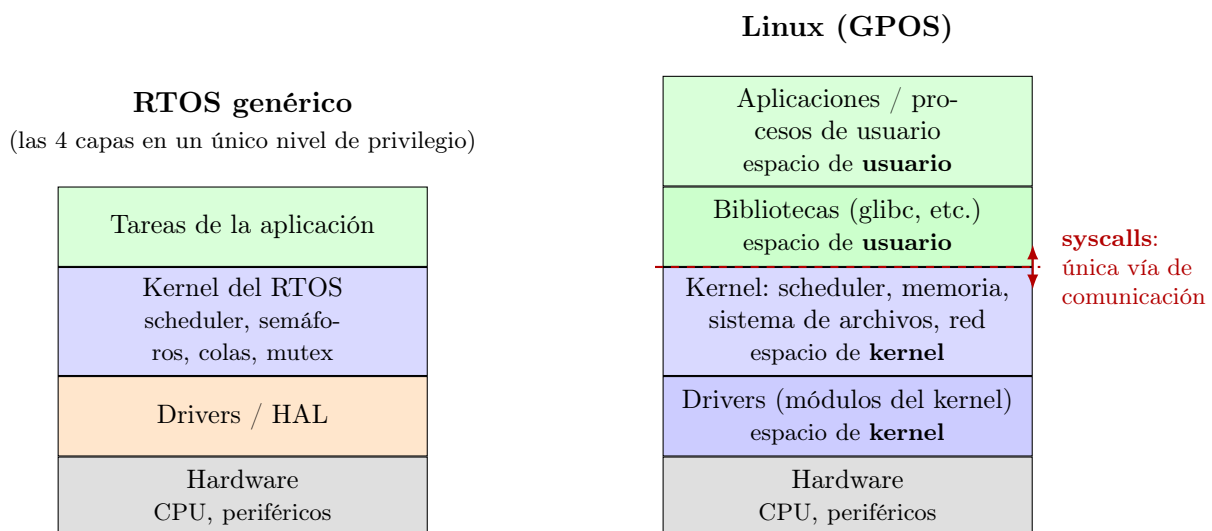


Figura 1: Capas de abstracción desde la aplicación hasta el hardware, comparando un RTOS genérico con Linux. En el RTOS, las cuatro capas corren en un único nivel de privilegio. En Linux, hay una frontera de privilegio real entre el espacio de kernel y el espacio de usuario, impuesta por la MMU, que solo se cruza a través de llamadas al sistema (*syscalls*).

Recorriendo la pila de un RTOS genérico, de abajo hacia arriba:

- **Hardware:** la CPU y los periféricos del microcontrolador (UART, GPIO, temporizadores, etc.).
- **Drivers / HAL:** código que accede directamente a los registros del hardware, muchas veces provisto por el propio fabricante del microcontrolador como parte de su SDK.
- **Kernel del RTOS:** el scheduler y las primitivas de sincronización (semáforos, colas, mutex) que ya conocen.
- **Tareas de la aplicación:** el código que escribe el desarrollador.

La clave es que estas cuatro capas se compilan y enlazan juntas en un único binario, corren en un único espacio de direcciones, y –lo más importante para esta comparación– **ninguna corre con más privilegio que las otras**: no hay un mecanismo de hardware que distinga “modo kernel” de “modo aplicación”. Un error en la capa de aplicación puede corromper perfectamente la memoria del kernel del RTOS, porque ambas viven en el mismo nivel.

En Linux, en cambio, cada capa corre con un privilegio definido, y eso sí está marcado en la Figura 1:

- **Hardware:** igual que antes, la CPU y los periféricos.
- **Drivers (módulos del kernel):** corren en **espacio de kernel**, con acceso completo al hardware. Es el equivalente más directo a la capa de drivers del RTOS, pero integrado al kernel de Linux en lugar de ser código aparte.
- **Kernel:** también en **espacio de kernel**. Acá viven el scheduler, la gestión de memoria, los sistemas de archivos, la pila de red, y todo lo demás que provee Linux como sistema operativo.
- **Bibliotecas (glibc, etc.):** ya en **espacio de usuario**. Son código que corre con los mismos privilegios limitados que cualquier aplicación; su rol es ofrecerle a la aplicación funciones cómodas (como `printf` o `malloc`) que, donde haga falta, se traducen internamente en una syscall.
- **Aplicaciones / procesos de usuario:** también en **espacio de usuario**, aislados además entre sí (un proceso no puede tocar la memoria de otro).

¿Cómo se hablan las dos mitades? Únicamente a través de una **syscall**: un proceso de usuario no puede leer un archivo, abrir un dispositivo, o crear otro proceso directamente, porque no tiene los privilegios de hardware para hacerlo. En cambio, le pide al kernel que lo haga por él, a través de una instrucción especial de la CPU que provoca un cambio controlado de modo usuario a modo kernel, ejecuta la operación pedida del lado del kernel, y devuelve el control (junto con un resultado) al proceso que la invocó. Esa instrucción –y no una llamada a función común– es la única puerta que cruza la frontera de privilegio marcada en la Figura 1. Las bibliotecas de espacio de usuario, como `glibc`, existen en buena medida para esconder ese detalle: cuando un programa llama a `read()`, en realidad está llamando a una función de la biblioteca que, por debajo, ejecuta la syscall correspondiente.

3.3. Tabla comparativa

La Tabla 1 resume los puntos de contacto más directos entre lo que el alumno ya conoce de un RTOS genérico y sus equivalentes (no siempre exactos) en Linux.

Un punto importante: Linux *sí* ofrece políticas de scheduling de tiempo real (`SCHED_FIFO` y `SCHED_RR`), que permiten asignarle a un proceso una prioridad fija y que se ejecute antes que cualquier proceso de tiempo compartido [1, cap. 17]. La diferencia con un RTOS es que en Linux

Concepto	RTOS genérico	Linux (GPOS)
Unidad de ejecución	Tarea, definida por el desarrollador con su propia pila y prioridad fija	Proceso / hilo (<code>pid</code> , <code>pthread_t</code>)
Scheduler (planificador)	Por prioridad fija, cooperativo o preemptivo, predecible	CFS (<i>Completely Fair Scheduler</i>) por defecto: prioriza equidad entre procesos, no determinismo [1, cap. 17]
Prioridades	Niveles fijos definidos en el diseño	Política de tiempo compartido (<code>SCHED_OTHER</code>) o políticas de tiempo real (<code>SCHED_FIFO</code> , <code>SCHED_RR</code>) [1, cap. 17]
Memoria	Un único espacio de direcciones, sin protección entre tareas	Memoria virtual, con protección entre procesos mediante la MMU
Binario resultante	Una sola imagen monolítica (aplicación + kernel del RTOS)	Kernel + múltiples binarios independientes + bibliotecas dinámicas
Determinismo	Alto, por diseño	Bajo por defecto; mejorable con <code>PREEMPT_RT</code>

Tabla 1: Comparación conceptual entre un RTOS genérico y Linux estándar.

esto es la excepción, no la regla: la mayoría de los procesos del sistema corren bajo la política por defecto (`SCHED_OTHER`), que prioriza equidad y *throughput* general por sobre la latencia de un proceso en particular.

3.4. Políticas de scheduling, en más detalle

Vale la pena detenerse en esto, porque es uno de los puntos donde un RTOS y Linux se parecen más de lo que parece a primera vista. La mayoría de los RTOS (incluido el que ya conocen) ofrecen, típicamente, dos variantes de scheduling:

- **Prioridad fija preemptiva:** la variante más común. Cada tarea tiene una prioridad asignada en el diseño; la tarea de mayor prioridad que esté lista para correr es la que corre, y una tarea de mayor prioridad que se desbloquea interrumpe (*preempts*) a una de menor prioridad que esté corriendo.
- **Round-robin entre tareas de igual prioridad:** cuando dos o más tareas tienen la misma prioridad, muchos RTOS reparten el tiempo de CPU entre ellas con *time slicing* (cada una corre durante una porción de tiempo fija, por turnos).

Algunos RTOS también permiten un modo **cooperativo**, donde una tarea corre hasta que ella misma decide cederle el control a otra (sin que el scheduler la interrumpa), pero ese modo es cada vez menos común en proyectos nuevos.

Linux, en lugar de elegir una sola de estas variantes, las ofrece todas como políticas distintas que se asignan por proceso (o por hilo). La Tabla 2 las resume [1, cap. 17].

La correspondencia con lo que ya conocen de un RTOS es bastante directa: `SCHED_FIFO` y `SCHED_RR` son, conceptualmente, las mismas dos variantes (prioridad fija preemptiva, y prioridad fija con round-robin) que ya usaron, solo que en Linux conviven con las políticas de tiempo compartido en lugar de ser la única opción disponible. Para asignarle a un hilo una política de tiempo real hace falta el privilegio `CAP_SYS_NICE`, normalmente reservado al superusuario, justamente porque un hilo de tiempo real mal diseñado (por ejemplo, con un bucle infinito por error) puede dejar sin CPU a todo el resto del sistema [1, cap. 17].

Política	Tipo	Prioridad	Entre tareas de igual prioridad
SCHED_OTHER (SCHED_NORMAL)	Tiempo compartido (CFS); la política por defecto	<i>nice</i> : -20 a 19	Reparto proporcional de CPU según el peso de cada hilo, no hay turnos fijos
SCHED_BATCH	Tiempo compartido	<i>nice</i> : -20 a 19	Igual que SCHED_OTHER, pero con cambios de contexto menos frecuentes (pensada para trabajos en segundo plano)
SCHED_IDLE	Tiempo compartido, la más baja posible	–	Solo corre cuando no hay ningún otro hilo listo, de ninguna otra política
SCHED_FIFO	Tiempo real, prioridad fija	1 a 99	Corre hasta bloquearse, cederse, o ser desalojado por uno de mayor prioridad (orden FIFO, sin <i>time slicing</i>)
SCHED_RR	Tiempo real, prioridad fija	1 a 99	Igual que SCHED_FIFO, pero con <i>time slicing</i> round-robin (100 ms por defecto) entre ellas

Tabla 2: Políticas de scheduling disponibles en Linux [1, cap. 17].

4. Los cuatro elementos de un sistema Linux embebido

Todo proyecto de Linux embebido se construye, en mayor o menor medida, a partir de cuatro elementos, ilustrados en la Figura 2: el toolchain, el bootloader, el kernel y el sistema de archivos raíz [1, cap. 1]. En Raspberry Pi OS estos cuatro elementos ya vienen preparados y combinados para nosotros, pero es importante entender qué hace cada uno, porque en los módulos siguientes vamos a tocar directamente el kernel (compilando y cargando módulos) y el sistema de archivos (creando nodos de dispositivo).

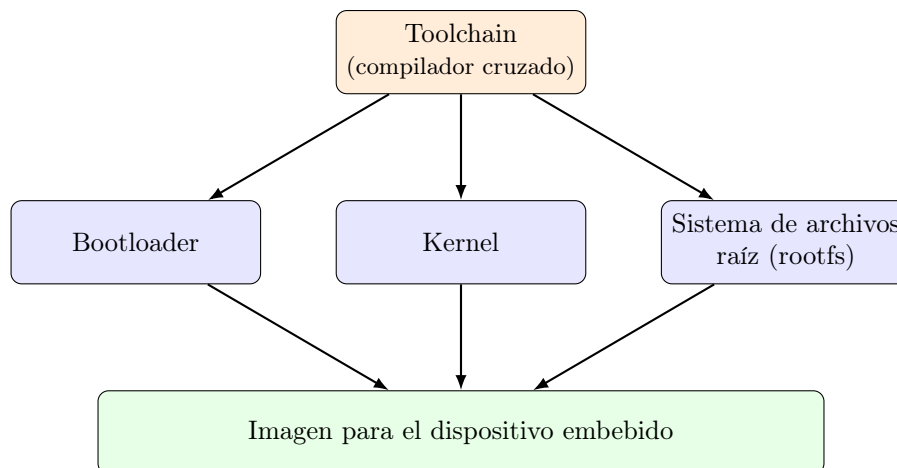


Figura 2: Los cuatro elementos de un sistema Linux embebido (adaptado de [1, cap. 1]).

Toolchain. Es el conjunto de herramientas (compilador, ensamblador, enlazador, depurador) que permite generar código ejecutable para una arquitectura de CPU distinta de la de la máquina donde se desarrolla, lo que se llama *compilación cruzada* (*cross-compilation*) [1, cap. 2]. Incluye también la biblioteca de C (*glibc*, *musl*, etc.) contra la que se enlazan los programas.

Bootloader. Es el primer programa que se ejecuta cuando se enciende el dispositivo. Su trabajo es inicializar el mínimo de hardware necesario (memoria RAM, reloj del sistema) y cargar el kernel en memoria para entregarle el control [1, cap. 3]. En la Raspberry Pi, buena parte de este rol lo cumple el firmware propio de la placa (almacenado en una EEPROM), que es justamente lo que se encarga de cargar el kernel de Linux al encender el equipo.

Kernel. Es el núcleo del sistema operativo: gestiona el hardware (CPU, memoria, dispositivos), provee las abstracciones centrales del sistema (procesos, archivos, redes) y expone una interfaz al espacio de usuario a través de llamadas al sistema (*syscalls*) [1, cap. 4]. Es la pieza que vamos a extender con un módulo propio en el Módulo 2.

Sistema de archivos raíz (rootfs). Es el conjunto de programas, bibliotecas y archivos de configuración que corren en espacio de usuario una vez que el kernel terminó de arrancar. Es, en definitiva, lo que vemos cuando navegamos con `ls` / [1, cap. 5].

5. Preparación del entorno: Raspberry Pi OS Lite headless

A partir de acá trabajamos en modo totalmente práctico. La idea es dejar la Raspberry Pi funcionando sin necesidad de monitor, teclado ni mouse, algo central en cualquier proyecto embebido real. Los pasos siguientes están tomados de la documentación oficial de Raspberry Pi [2], ya que cubren una herramienta específica del fabricante.

5.1. Paso 1: Grabar la imagen con Raspberry Pi Imager

1. Descargar *Raspberry Pi Imager* desde <https://www.raspberrypi.com/software/>.
2. Insertar la microSD en el lector y abrir Imager.
3. Elegir el modelo de placa (Raspberry Pi 4 o 5, según corresponda).
4. En sistema operativo, elegir **Raspberry Pi OS Lite** (sin entorno de escritorio: no lo necesitamos para un sistema headless).
5. Elegir el almacenamiento (la microSD insertada).
6. Antes de grabar, hacer clic en el ícono de engranaje (opciones avanzadas de personalización del sistema operativo) y configurar:
 - **Hostname:** por ejemplo `rpi-lse-01`.
 - **Usuario y contraseña:** ya no existe el usuario `pi` por defecto; hay que crear un usuario propio.
 - **Wi-Fi:** SSID (nombre de la red) y contraseña, si no se va a usar cable de red.
 - **Habilitar SSH** (*Secure Shell*, el protocolo que vamos a usar para conectarnos): marcar la opción de autenticación por contraseña (o por clave pública, si el alumno ya tiene un par de claves SSH generado).
7. Guardar la configuración y confirmar la escritura de la imagen.

Este mecanismo de pre-configuración es la forma recomendada actualmente por Raspberry Pi para evitar tener que conectar un monitor, aunque sea una sola vez [2].

5.2. Paso 2: Conectarse por SSH desde la PC

El cliente de SSH viene incluido por defecto en Windows (10 build 1809 en adelante, y Windows 11), macOS y la gran mayoría de las distribuciones de Linux. En los tres casos alcanza con abrir una terminal (PowerShell o el Símbolo del sistema en Windows, Terminal en macOS, el emulador de terminal que prefieran en Linux) y ejecutar el mismo comando:

1. Insertar la microSD en la Raspberry Pi y conectar la alimentación.
2. Esperar 1–2 minutos: el primer arranque tarda más porque la Pi expande el sistema de archivos y aplica la configuración de red.
3. Conectarse usando el hostname configurado:

```
ssh lse@lse-pi4-xx.local
```

4. Reemplazar xx por el número de la Raspberry Pi asignada.
5. Si el nombre `.local` no resuelve (puede pasar en Windows sin soporte mDNS), buscar la IP asignada por el router o escanear la red:

```
ssh lse@192.168.1.XX
```

5.3. Paso 3: Primeros comandos de chequeo

```
# Información del sistema
uname -a
cat /etc/os-release

# Espacio en disco y memoria
df -h
free -h

# Actualizar el sistema
sudo apt update && sudo apt full-upgrade -y
```

6. El sistema de archivos de Linux

6.1. Permisos de archivos

Linux organiza todo –archivos, dispositivos, procesos en ejecución– como un único árbol de archivos a partir de la raíz `/`. Cada archivo tiene un dueño, un grupo, y un conjunto de permisos de lectura, escritura y ejecución para el dueño, el grupo, y el resto de los usuarios.

```
ls -l /etc/hostname
# -rw-r--r-- 1 root root 11 Jun 1 10:00 /etc/hostname
```

En el ejemplo, `rw-` son los permisos del dueño (lectura/escritura), `r-` los del grupo (solo lectura) y `r-` los del resto (solo lectura). Esto va a ser relevante más adelante: cuando un módulo de kernel crea un dispositivo de carácter en `/dev`, los permisos de ese nodo determinan qué usuarios pueden abrirlo.

6.2. Directorios más relevantes

La Figura 3 muestra, de forma esquemática, cómo se relacionan entre sí los directorios más importantes a partir de la raíz. La Tabla 3 después detalla para qué se usa cada uno.

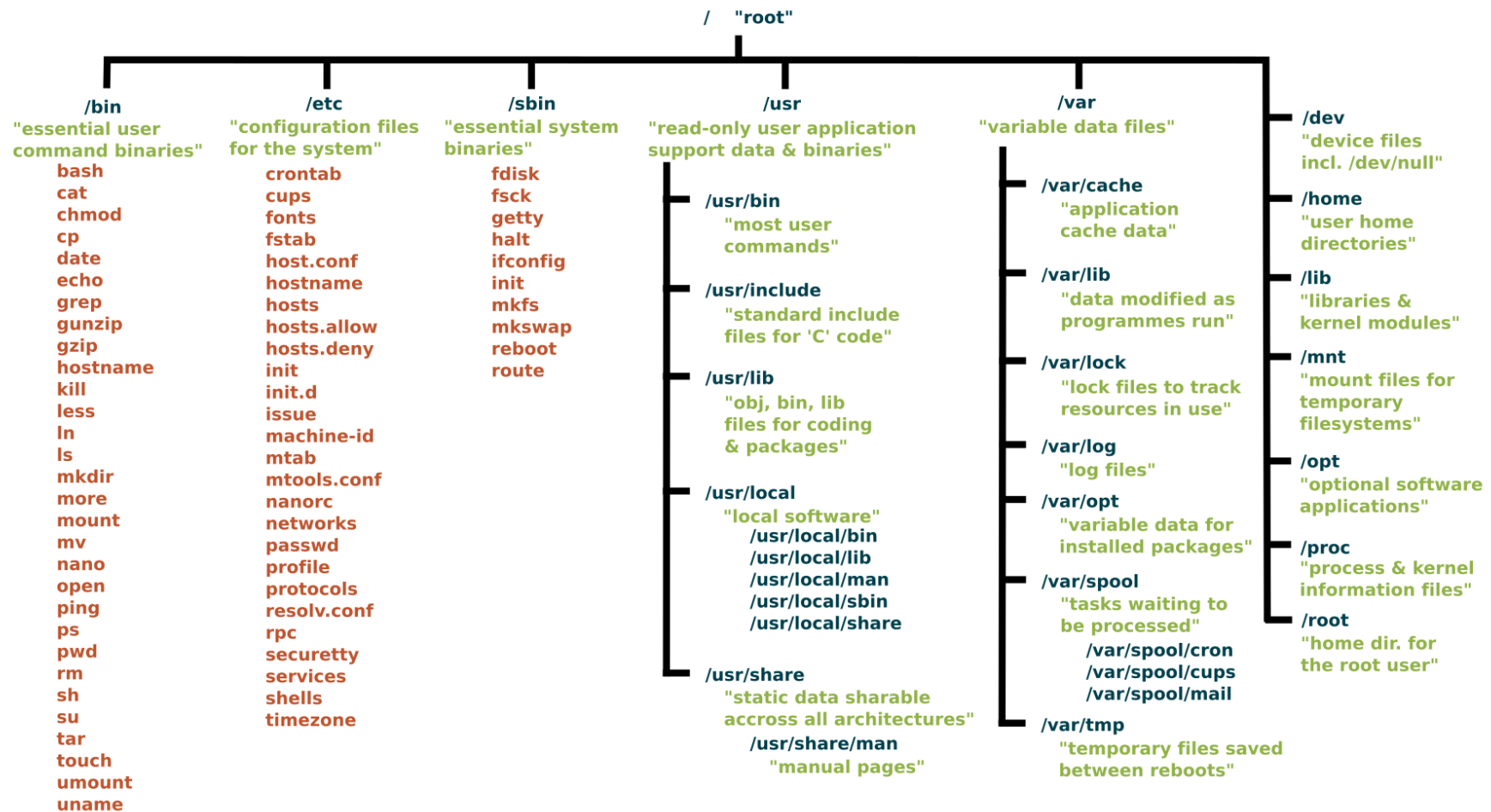


Figura 3: Jerarquía estándar de directorios de Unix/Linux. Fuente: Linux Foundation, <https://www.linuxfoundation.org/>.

La Tabla 3 resume los directorios con los que vamos a interactuar más seguido a lo largo del curso [1, cap. 5].

Directorio	Uso
<code>/bin</code> , <code>/usr/bin</code>	Programas binarios esenciales del sistema (en distribuciones modernas, <code>/bin</code> suele ser un enlace simbólico a <code>/usr/bin</code>).
<code>/sbin</code> , <code>/usr/sbin</code>	Binarios de administración del sistema, normalmente pensados para usarse con privilegios de superusuario.
<code>/etc</code>	Archivos de configuración del sistema y de los servicios instalados.
<code>/home</code>	Directorios personales de los usuarios (por ejemplo, <code>/home/usuario</code>).
<code>/root</code>	Directorio personal del superusuario <code>root</code> .
<code>/var</code>	Datos que cambian con el tiempo: logs, colas de impresión, cachés.
<code>/tmp</code>	Archivos temporales; suele vaciarse en cada reinicio.
<code>/dev</code>	Nodos de dispositivo: cada periférico accesible aparece acá como un archivo especial. Vamos a crear nuestros propios nodos en <code>/dev</code> a partir del Módulo 4.
<code>/proc</code>	Pseudo-sistema de archivos que expone información en tiempo real sobre los procesos y el estado del kernel. No lo vamos a usar todavía, pero existe y vale la pena conocerlo.
<code>/sys</code>	Pseudo-sistema de archivos que expone el modelo de dispositivos del kernel. Tampoco lo vamos a usar todavía, pero reaparece más adelante en el curso.
<code>/lib</code> , <code>/usr/lib</code>	Bibliotecas compartidas necesarias para que los programas de <code>/bin</code> y <code>/sbin</code> funcionen.
<code>/opt</code>	Software opcional o de terceros, instalado fuera del sistema de paquetes estándar.
<code>/mnt</code> , <code>/media</code>	Puntos de montaje para sistemas de archivos externos o removibles (pendrives, discos).
<code>/boot</code>	Archivos necesarios para el arranque: el kernel y, en otras plataformas, el bootloader.

Tabla 3: Directorios principales del sistema de archivos de Linux.

7. Comandos esenciales de Linux

La Tabla 4 es una referencia rápida de los comandos que vamos a necesitar seguido durante el curso. No hace falta memorizarlos: la idea es tenerlos a mano y volver a esta tabla cada vez que sea necesario.

Comando	Para qué sirve
<i>Navegación y archivos</i>	
<code>pwd</code>	Muestra el directorio actual.
<code>ls -l</code>	Lista archivos con detalle (permisos, tamaño, fecha).
<code>cd <directorio></code>	Cambia de directorio.
<code>mkdir <directorio></code>	Crea un directorio.
<code>cp origen destino</code>	Copia un archivo.
<code>mv origen destino</code>	Mueve o renombra un archivo o directorio.

Comando	Para qué sirve
<code>rm archivo</code> (<code>rm -r</code> para directorios)	Borra un archivo o directorio.
<code>cat archivo</code>	Muestra el contenido completo de un archivo.
<code>less archivo</code>	Muestra el contenido de un archivo, paginado (salir con <code>q</code>).
<code>nano archivo</code>	Editor de texto simple en la terminal.
<i>Búsqueda</i>	
<code>grep "texto.archivo"</code>	Busca un patrón de texto dentro de un archivo.
<code>find /ruta -name "*.txt"</code>	Busca archivos por nombre a partir de una ruta.
<i>Permisos y usuarios</i>	
<code>chmod 755 archivo</code>	Cambia los permisos de un archivo.
<code>chown usuario:grupo archivo</code>	Cambia el dueño y/o grupo de un archivo.
<code>sudo comando</code>	Ejecuta un comando con privilegios de superusuario (<code>root</code>).
<i>Información del sistema</i>	
<code>uname -a</code>	Muestra información del kernel y la arquitectura de CPU.
<code>df -h</code>	Muestra el espacio en disco usado y disponible.
<code>free -h</code>	Muestra la memoria RAM usada y disponible.
<code>ps aux</code>	Lista los procesos en ejecución, con su PID (<i>Process ID</i> , identificador de proceso).
<code>top</code>	Muestra el uso de CPU y memoria en tiempo real.
<i>Red y acceso remoto</i>	
<code>ssh usuario@host</code>	Conecta por SSH a otro equipo.
<code>scp archivo usuario@host:/ruta</code>	Copia archivos hacia o desde otro equipo a través de SSH.
<i>Gestión de paquetes (Debian / Raspberry Pi OS)</i>	
<code>sudo apt update</code>	Actualiza la lista de paquetes disponibles.
<code>sudo apt install <paquete></code>	Instala un paquete.
<i>Kernel (lo vamos a usar a partir del Módulo 2)</i>	
<code>dmesg</code>	Muestra los mensajes del kernel (buffer circular de log).

Tabla 4: Comandos esenciales de Linux para el curso.

8. Cierre conceptual

Con la Pi accesible, los conceptos de RTOS vs. GPOS establecidos, y el sistema de archivos ya familiar, el resto del bloque va a ir construyendo, capa por capa, las piezas necesarias para que esta Raspberry Pi termine comunicándose por UART con una placa ESP32-S3 corriendo FreeRTOS: módulos de kernel, sincronización con hilos, dispositivos de carácter y, finalmente, un *device tree overlay* a medida.

Referencias

- [1] Vasquez, F. y Simmonds, C. (2021). *Mastering Embedded Linux Programming*, 3.^a edición. Packt Publishing.
- Cap. 1, *Starting Out* – elección de Linux, los cuatro elementos del Linux embebido.
 - Cap. 2, *Learning about Toolchains* – compilación cruzada y toolchains.
 - Cap. 3, *All about Bootloaders* – qué hace un bootloader.
 - Cap. 4, *Configuring and Building the Kernel* – qué hace el kernel.
 - Cap. 5, *Building a Root Filesystem* – estructura de directorios, permisos de archivos.
 - Cap. 17, *Learning about Processes and Threads* – políticas de scheduling.
 - Cap. 21, *Real-Time Programming* – determinismo, latencia de scheduling, PREEMPT_RT.
- [2] Raspberry Pi Ltd. *Getting started – Raspberry Pi Documentation*. <https://www.raspberrypi.com/documentation/computers/getting-started.html> (consultado en 2026).